Lab W11: The Guild Commission Board, Part 2

CIS 3300, Intermediate Object-Oriented Programming

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

A note on working before your review is back

You will start this lab before you receive feedback on Lab W10. That is normal in software development: developers often keep building on their own work while an earlier change is still waiting for review. Google's published guide for its own engineers puts it plainly: "So you want to find some way to work that won't block you while you're waiting for review." Work from your own best judgment. If your Lab W10 feedback arrives while you are working, use it, and explain what you changed in a `DECISION (changed):` comment.

Source: Google Engineering Practices Documentation, "Small CLs".

---

1 The Story

The guildmaster liked your board report. Now the Coppertop Guild Hall wants to hand out the work. The clerk keeps a roster of the guild's members, and each member has a name, a rank, and a specialty. Some members also hold a license to handle valuable goods. The ranks, from lowest to highest, are copper, silver, and gold.

1.1 Who may take a commission

A member may take a commission only if the member's rank is the same as or higher than the commission's rank. Some kinds of commissions add a rule of their own, and some kinds of work are hazardous. A member who completes hazardous work receives a hazard bonus. The client pays the bonus on top of the payout, and all of it goes to the member: the guild takes no cut of it. Table 1 lists the rules for each kind.

Table 1: The guild's rules for each kind of commission.

Kind	Who may take it, besides the rank rule	Hazard bonus
Hunt	A member whose specialty is warrior, ranger, or battlemage	5 gold per threat level, once per hunt
Escort	Anyone	None
Delivery	Anyone	25 gold if the route passes through dangerous country, otherwise none
Retrieval	A licensed member, if the item is worth 1000 gold or more; anyone otherwise	None

1.2 How the work is handed out

The clerk goes through the board in order. Each commission goes to the eligible member who has been given the fewest commissions so far. If several eligible members are tied, the one listed first in the roster gets it. If no member is eligible, the commission stays unassigned.

1.3 Duplicate postings

The clerk sometimes posts the same commission twice. A commission posted twice has the same ID and the same details both times. It must appear only once in every report, and it is handed out only once.

2 Project Setup

1. Create a new Java project named `labw11`, and copy your `guild` and `app` packages from `labw10` into it. Leave your Lab W10 project as it is.
2. Set up anything the project needs to read the files, the same way as in Lab W10.
3. Keep the same packages: the classes that represent commissions and members in `guild`, and your program with `main` in `app`.
4. Create a `files` folder in the project's root folder. Place the new `board` file, the `roster` file, and both sample reports in it, and write both of your reports to that same folder. The board files for this part replace the ones from Lab W10.

3 What You Are Building

Start from your Lab W10 code and change it. Do not start over: a good part of this assignment is seeing how well your Lab W10 design holds up when the requirements change.

- **Classes that represent the members** on the roster, along with any changes your commission classes need for the new rules.
- **The board report from Lab W10**, updated. It no longer lists a duplicate posting twice, it shows each commission's hazard bonus, and its totals include the total of all hazard bonuses.
- **A new report** in a text file named `assignments_report.txt`. For each member, in roster order, it lists the commissions that member was given, with the member's share, the hazard

bonus, and what the member earns from each, followed by the member's total earnings. A member who was given nothing is still listed. The report ends with every commission that no member could take.

Your program reads the board file and the roster file, the way we did in lecture, and writes both reports, the way we did in lecture. Deliveries in the new board files have one new field, `dangerous`, that tells whether the route passes through dangerous country.

The sample reports for the `board` and `roster` files, `sample_board_report.txt` and `sample_assignments_report.txt`, show the layout your reports must follow. As in Lab W10, match them closely: the same sections, in the same order, with the same information. Exact spacing may vary within reason, but both reports must stay clean and aligned for every board file.

You may add any other fields, methods, or classes you find useful.

4 Many Decisions Are Yours

This assignment deliberately does not tell you which classes to write, what to name them, or how they relate to each other. You decide, and you will need to be able to explain why. This does not mean that "anything goes" for points.

There is more than one good answer, and your design does not need to match a classmate's. What matters is that your decisions are deliberate, consistent, and match what your code actually does.

Your decisions and your code must be consistent with what we have covered through Week 11, in both the lectures and the assigned book chapters; this assignment is about applying that knowledge. Your design must use inheritance and polymorphism in a meaningful way, including in how it handles the new rules. A design that avoids them, or uses them only on the surface, does not meet the expectations for this assignment.

5 Design Vocabulary

These are the same terms as in Lab W10. You are expected to know what each one means and to find out more about it on your own, using the textbook and other reliable sources.

- Encapsulation
- Cohesion
- Coupling
- Reusability
- Extensibility
- Substitutability

6 Explaining Your Decisions

As in Lab W10, you explain your design in comments in your code, written in your own words. Selected students may be asked to walk through these comments in a personal review, as described in the syllabus.

- **Your decisions.** Explain every major design decision you made, in a comment placed right next to the code where you made it, starting with `DECISION:`. This includes decisions you kept from Lab W10.
- **Decisions you changed.** If you changed a decision from Lab W10, update its comment to start with `DECISION (changed):` and say what the decision was, what it is now, and why it changed. Changing a decision is not a mistake. It is what designers do when the requirements change.
- **Design terms.** You need three `DESIGN TERM:` comments in all:
  1. **Your original term from Lab W10.** Keep it, and update it so it still matches the code beside it.
  2. **A second term on the same code.** Place it right beside your original comment, and use a different term to explain that same code.
  3. **One term for the new parts of this lab.** Place it beside code you wrote for this lab: the members, the rules for who may take a commission, hazard bonuses, duplicate postings, or the assignments report. Any term is fine, even one you have already used.

  Comments 2 and 3 take the same form as in Lab W10: your design, a weaker design, and a better design with its cost.

The commenting rules from Lab W10 still apply: each file needs the general comment at the top, and beyond that, put your effort into comments that explain your design.

6.1 Examples

These examples continue the coffee shop from Part 1. Part 2 of that program added a second receipt for the kitchen.

```
// DECISION (changed): in Part 1, main printed the one receipt itself.
// With a second receipt for the kitchen, each receipt has its own
// method, so main reads as the steps of taking an order.
private static void printCustomerReceipt(List<Drink> order) {
    ...
}
```

The original design term from Part 1, updated, with a second term beside it on the same code:

```
// DESIGN TERM: [original term]
// My design: Lorem ipsum dolor sit amet, consectetur adipiscing elit,
//   sed do eiusmod tempor incididunt ut labore.
// Weaker: Ut enim ad minim veniam, quis nostrud exercitation ullamco
//   laboris nisi ut aliquip ex ea commodo.
// Better, at a cost: Duis aute irure dolor in reprehenderit in voluptate
//   velit esse. Cost: cillum dolore eu fugiat nulla pariatur.
// DESIGN TERM: [second term]
// My design: Excepteur sint occaecat cupidatat non proident, sunt in
//   culpa qui officia deserunt mollit.
// Weaker: Sed ut perspiciatis unde omnis iste natus error sit
//   voluptatem accusantium doloremque.
// Better, at a cost: Nemo enim ipsam voluptatem quia voluptas sit
//   aspernatur. Cost: aut odit aut fugit.
for (Drink d : order) {
    total += d.price();
}
```

A design term for something new in Part 2, the kitchen ticket:

```
// DESIGN TERM: [any term]
// My design: Lorem ipsum dolor sit amet, consectetur adipiscing elit.
// Weaker: Ut enim ad minim veniam, quis nostrud exercitation.
// Better, at a cost: Duis aute irure dolor in reprehenderit. Cost:
//   excepteur sint occaecat cupidatat.
private static void printKitchenTicket(List<Drink> order) {
    ...
}
```

7 Additional Sample Data

New versions of `board2` and `board3` are provided for testing, and both are used with the same `roster` file. Place them in your `files` folder. To test with one of them, change the board file name your program reads and run it again. The program you submit should read `board` and `roster`.

8 Requirements Checklist

- Your Lab W10 design is reworked, not replaced, and it still uses inheritance and polymorphism in a meaningful way, along with encapsulation and the other principles of good class design we discussed in class.
- Members are represented by classes of your own design.
- Hazard bonuses, the rules for who may take each commission, and the way work is handed out all follow the guild's rules.
- A duplicate posting appears only once in every report and is handed out only once.
- Your program reads the board file and the roster file the way we did in lecture.

- Your program writes both reports to text files the way we did in lecture, closely following the layout of the sample reports, and they stay clean and aligned for every board file.
- Your `DECISION:` comments explain every major design decision next to the code that carries it out, your `DECISION (changed):` comments explain what changed since Lab W10 and why, and your code matches what they say.
- You have three `DESIGN TERM:` comments: your Lab W10 term, updated to match the code; a second term beside it that explains the same code; and one term for the new parts of this lab. Comments 2 and 3 each describe your design, a weaker design, and a better design with its cost.
- Your code meets the course's standards for code quality, and every file has the required header comment.

9 Grading

Items in Tables 2 and 3 are worth 1 point each and are either met or not met. Items in Table 4 are worth a range of points, with partial credit based on how well your work meets the standards we set in lecture. Because there is more than one good design for this assignment, grading of the Table 4 items includes some subjectivity.

Table 2: Submission and setup (1 point each)

Item	Points
Submitted as a reasonable attempt that compiles and runs in Eclipse without errors	1
Project laid out as described in Project Setup (project name, packages, and `files` folder)	1

Table 3: Code quality (1 point each)

Item	Points
Consistent naming conventions used throughout	1
Consistent formatting and indentation throughout	1
No unrequested extra output	1
Each file has the required header comment	1
Loop and method logic reads clearly from top to bottom	1

Table 4: Design and implementation (partial credit possible)

Item	Points
The commission classes are updated well for the new rules	8
The member classes meet expectations	5
Commissions are handed out according to the guild's rules	6
The board and roster files are read using the approach from lecture	3
Both reports are complete and written using the approach from lecture	5
Design decisions, including changed ones, are explained in the code, and the code matches the explanations	8
The Lab W10 design term is updated, and a second term explains the same code	6
One design term is discussed for the new parts of the program	6

Total: 54 points.

10 Submission

Submit through your course GitHub repository, per the syllabus's submission guidelines. Include your `src` folder with the `guild` and `app` packages, and the `files` folder with the board and roster files. It is fine either way whether your report files are included, since your program creates them every time it runs.